

College Management System

Relational Database Project Report (`COLLEGEDB`)

Project Domain:	Higher Education Academic Administration	Database Engine:	MySQL / MariaDB
Schema Architecture:	Third Normal Form (3NF)	Primary Modules:	Departments, Students, Faculty, Courses, Enrollment

1. EXECUTIVE SUMMARY

1.1 Project Overview

The **College Management System** (`COLLEGEDB`) is a centralized relational database architecture engineered to store, organize, and streamline academic and administrative records within an educational institution. The system establishes a structured environment for managing multi-department operations, student profiles, faculty employment records, course catalogs, and academic enrollment performance.

1.2 Primary Objectives

- Data Normalization:** Structure data according to 3rd Normal Form (3NF) guidelines to eliminate redundant data storage and prevent insertion, update, and deletion anomalies.
- Referential & Entity Integrity:** Enforce relational consistency using primary keys, auto-incrementing surrogate identifiers, and foreign key constraints across all entity boundaries.
- Domain & Business Validation:** Implement explicit check constraints and strict data typing to validate student eligibility, grading parameters, and unique contact identities.
- Institutional Reporting:** Provide a foundational platform for administrative analytics, grade tracking, faculty payroll processing, and course load evaluations.

2. SYSTEM ARCHITECTURE & RELATIONAL SCHEMA

The system architecture models five core business entities. Relationships between entities are defined to reflect realistic institutional workflows, maintaining clean separation of concerns.

Department Parent entity representing physical & academic divisions across campus blocks.	Students Stores personal demographics, admission dates, and cumulative GPAs.	Faculty Tracks instructor profiles, compensation details, and employment start dates.	Courses Maintains catalog offerings associated with specific academic departments.	Enrollment Junction entity resolving M:N relationship between Students and Courses.
---	--	---	--	---

2.1 Entity Relationship Dynamics

- Department → Students (1:N):** One department admits multiple students; each student is assigned to a single parent department.
- Department → Faculty (1:N):** One department employs multiple faculty members; each faculty member belongs to one department.
- Department → Courses (1:N):** One department offers multiple academic courses; each course is owned by a single department.

- **Students • Courses → Enrollment (M:N):** A student registers for multiple courses, and a course receives multiple student enrollments. The **`Enrollment`** entity resolves this many-to-many relationship, recording specific term semesters and evaluated letter grades.

3. DATA DICTIONARY

3.1 Entity: `Department`

Column Name	Data Type	Key	Nullability	Constraints & Description
department_id	INT UNSIGNED	PK	NOT NULL	AUTO_INCREMENT. Primary key identifier.
department_name	VARCHAR(50)	-	NOT NULL	Official department title.
hod_name	VARCHAR(50)	-	NOT NULL	Full name of the Head of Department.
block	CHAR(1)	-	NOT NULL	Campus building/block code (e.g., 'A', 'B').

3.2 Entity: `Students`

Column Name	Data Type	Key	Nullability	Constraints & Description
student_id	INT UNSIGNED	PK	NOT NULL	AUTO_INCREMENT. Unique student roll number.
first_name	VARCHAR(50)	-	NOT NULL	Student's given name.
last_name	VARCHAR(50)	-	NULL	Student's surname.
email	VARCHAR(50)	UK	NULL	Unique email address constraint.
phone	VARCHAR(17)	UK	NULL	Unique contact phone number constraint.
age	TINYINT UNSIGNED	-	NULL	CHECK (age >= 17) eligibility validation.
gender	CHAR(1)	-	NULL	CHECK (gender IN ('M', 'F')) identity code.
CGPA	DECIMAL	-	NULL	DEFAULT 0.00. Academic performance rating.
DEPARTMENT_ID	INT UNSIGNED	FK	NULL	Ref: Department(department_id).
ADMISSION_DATE	DATE	-	NOT NULL	Official enrollment start date.

3.3 Entity: `Faculty`

Column Name	Data Type	Key	Nullability	Constraints & Description
faculty_id	INT UNSIGNED	PK	NOT NULL	AUTO_INCREMENT. Unique faculty ID.
faculty_name	VARCHAR(50)	-	NOT NULL	Full name of faculty instructor.
email	VARCHAR(50)	UK	NULL	Unique institutional email constraint.
salary	DECIMAL(10,2)	-	NOT NULL	Unsigned monthly/annual compensation.
department_id	INT UNSIGNED	FK	NULL	Ref: Department(department_id).
joining_date	DATE	-	NULL	DEFAULT (CURRENT_DATE). Hiring date.

3.4 Entity: `Courses`

Column Name	Data Type	Key	Nullability	Constraints & Description
course_id	INT	PK	NOT NULL	AUTO_INCREMENT. Primary key catalog code.
course_name	VARCHAR(30)	-	NOT NULL	Title of academic subject course.
department_id	INT UNSIGNED	FK	NULL	Ref: Department(department_id).

3.5 Entity: `Enrollment`

Column Name	Data Type	Key	Nullability	Constraints & Description
enrollment_id	INT UNSIGNED	PK	NOT NULL	AUTO_INCREMENT. Unique tracking ID.
student_id	INT UNSIGNED	FK	NULL	Ref: Students(student_id).
course_id	INT UNSIGNED	FK	NULL	Ref: Courses(course_id).
semester	TINYINT UNSIGNED	-	NULL	CHECK (semester BETWEEN 1 AND 8) limit.
grade	CHAR(2)	-	NULL	DEFAULT 'NA'. Awarded letter grade.

4. BUSINESS RULES & DATA INTEGRITY CONSTRAINTS

Institutional Rule Enforcement

- **Minimum Admission Age:** Enforces an age check (age >= 17) at table level to ensure compliance with higher education admission policies.
- **Contact Uniqueness Guarantee:** Enforces unique indexes on student and faculty email addresses and phone numbers to eliminate duplicate contact records.
- **Standardized Semester Boundaries:** Limits enrollment semesters strictly between 1 and 8, representing standard undergraduate program durations.
- **Default Initialization Rules:**
 - Newly admitted students default to a starting CGPA of 0.00.
 - Course registrations default to a grade of 'NA' until official faculty submission.
 - Faculty records default to system date (CURRENT_DATE) when no explicit joining date is provided.

5. SYSTEM ADVANTAGES & FUTURE ENHANCEMENTS

5.1 Architectural Advantages

The modular entity structure ensures scalability and data safety. By separating junction relationships (Enrollment) from core entities (Students, Courses), the schema allows seamless expansion without altering existing relational structures.

5.2 Future Roadmap

- **Cascading Referential Actions:** Implement `ON DELETE CASCADE` and `ON DELETE SET NULL` clauses to handle automatic cleanup of orphan enrollment records.
- **Audit Logging & Triggers:** Deploy database triggers on table updates (e.g., grade modifications and salary changes) to maintain an immutable compliance log.
- **Analytical Views:** Construct predefined database views for departmental performance metrics, faculty workload evaluations, and GPA distribution reports.

6. CONCLUSION

The `COLLEGEDB` database model establishes a clean, normalized, and highly reliable relational foundation tailored for higher education management. The design successfully eliminates operational redundancies, enforces business rules at the schema layer, and provides a clear path for future administrative application integration.